WeldTrack

System Documentation

Architecture · Data Model · Roles · Core Workflows

Generated from the application source · 2026-08-15

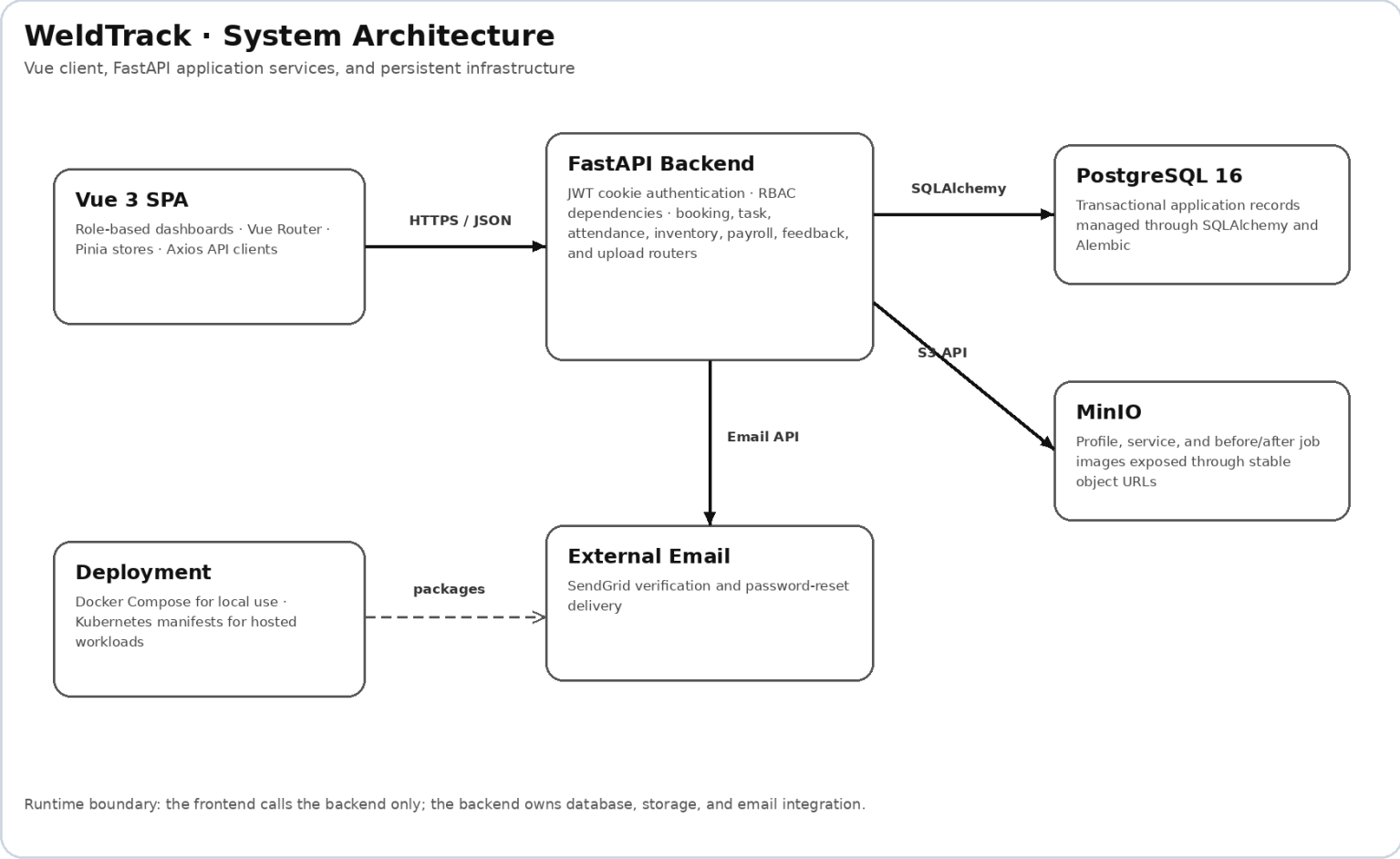
1. System overview

WeldTrack centralizes customer bookings, welding project delivery, staff coordination, attendance, inventory, payroll, and customer feedback. The application uses a Vue 3 single-page frontend and a FastAPI backend backed by PostgreSQL. Images are stored in MinIO, while SendGrid supports verification and password-recovery emails.

Primary users

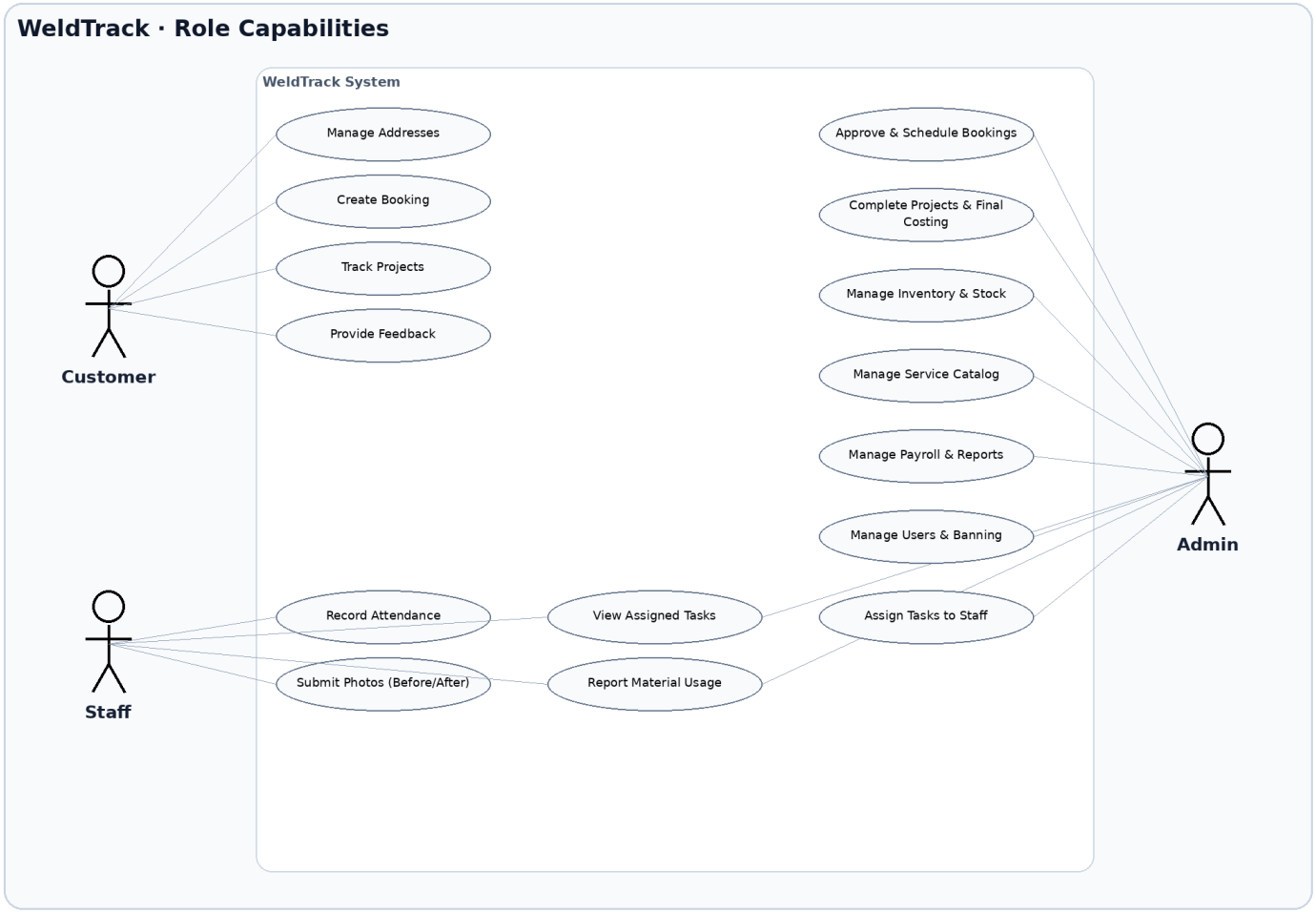
Role	Main responsibilities
Customer	Maintain addresses, create and track bookings, browse services, and submit feedback.
Staff	Record attendance, view assigned tasks, update task status, report materials and labor, and upload job photos.
Administrator	Approve and schedule bookings, manage users, services and inventory, assign work, complete costing, configure attendance, and manage payroll.

2. System architecture



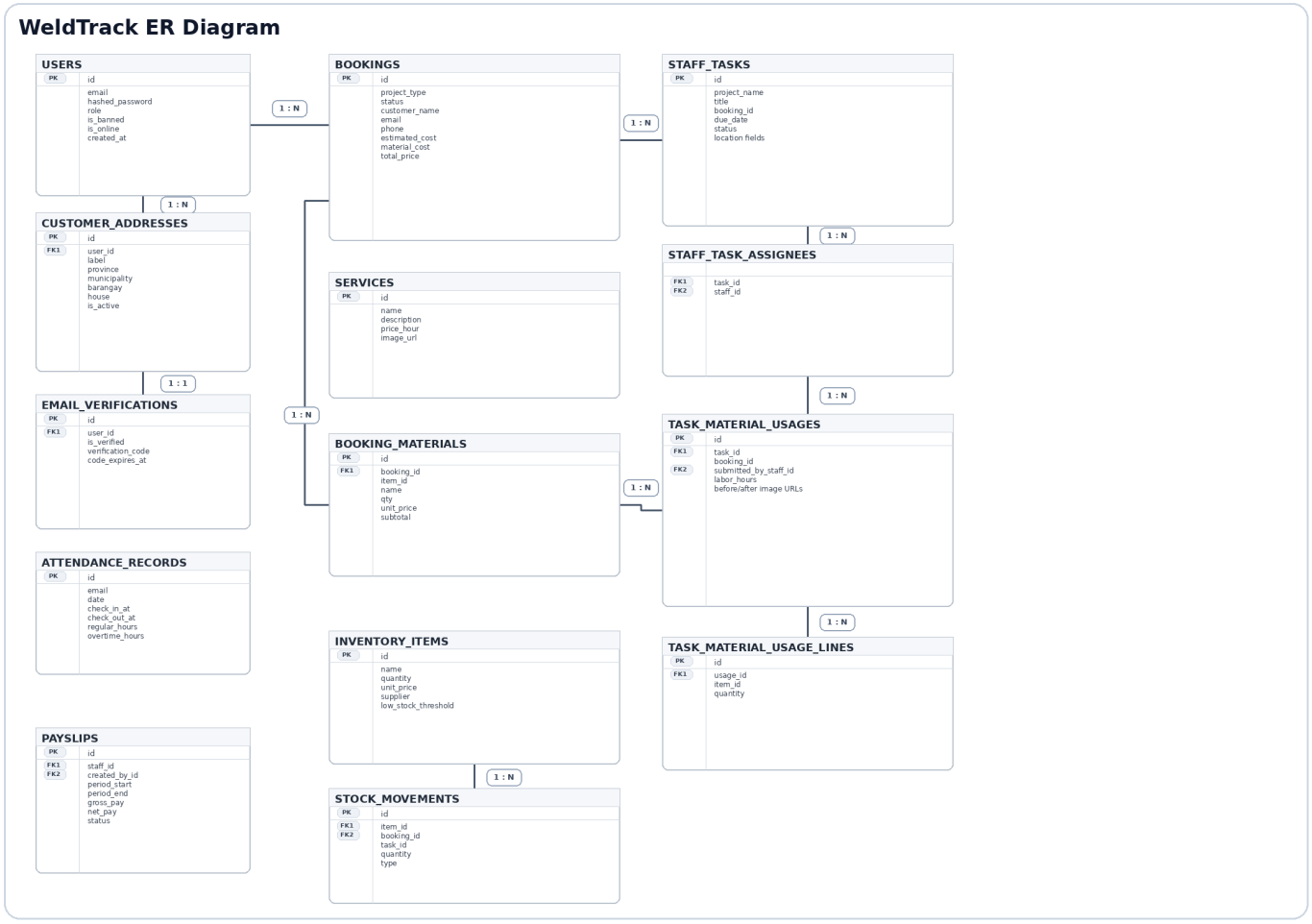
The browser communicates with the API; infrastructure integrations remain behind the backend boundary.

3. Role capabilities



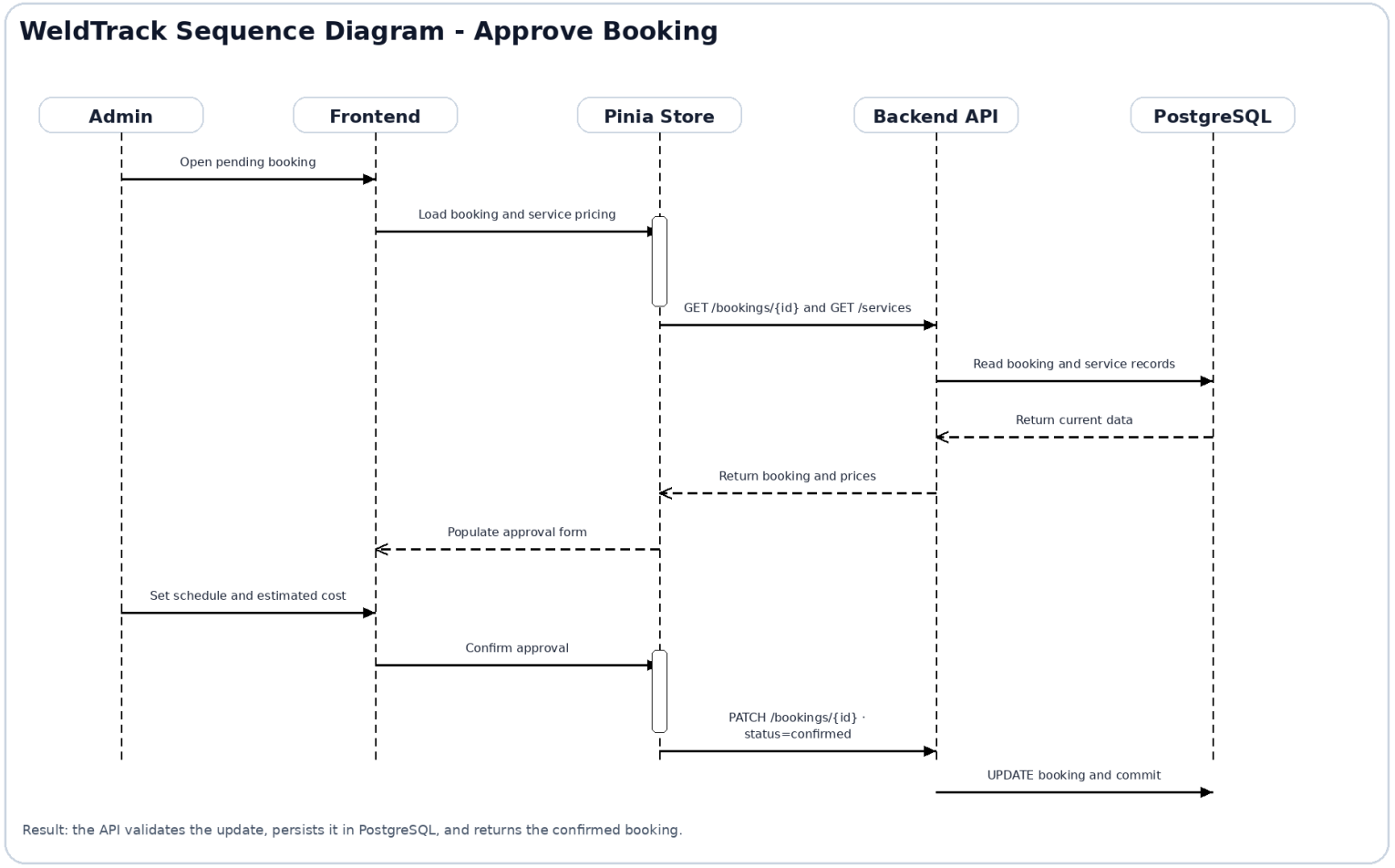
Capabilities are grouped by the role that initiates or manages each workflow.

4. Data model



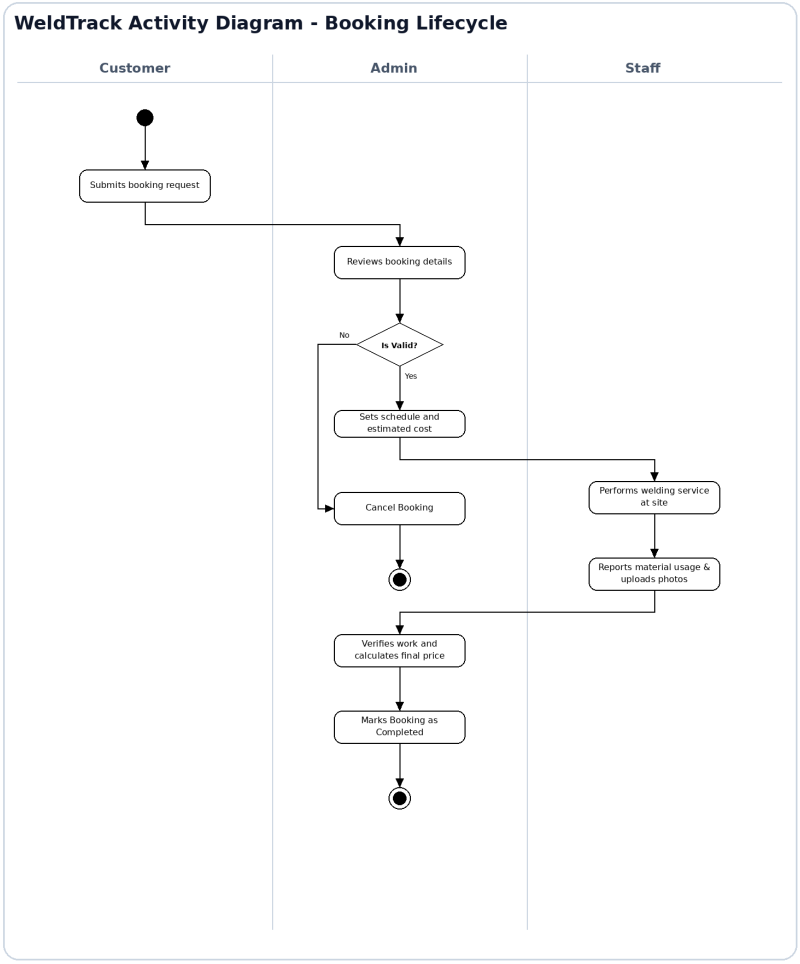
Core operational entities and implemented foreign-key relationships. Some loose references are intentionally shown as fields rather than enforced relationships.

5. Booking approval sequence



Administrator approval crosses the Vue view, Pinia store, FastAPI endpoint, and PostgreSQL persistence layers.

6. Booking lifecycle



The lifecycle separates customer submission, administrative control, and staff execution with explicit success and cancellation paths.

7. API and operational notes

API domains

The backend exposes routers for authentication and verification, profiles, customer addresses, bookings, services, inventory and stock movements, staff tasks and material usage, attendance, payroll, feedback, staff contributions, settings, administrative user management, and uploads.

Security model

Authentication uses JWT-based HTTP-only cookies. Route dependencies enforce role-based access. The application also applies CORS controls and a default request rate limit. Development API documentation is disabled when production mode is enabled.

Persistence and files

SQLAlchemy models use ULID identifiers and Alembic migrations against PostgreSQL. MinIO stores profile images, service images, and staff before/after work evidence. Application records retain object URLs rather than file bytes.

Deployment

Docker Compose defines frontend, backend, PostgreSQL, pgAdmin, and MinIO services for local or single-host deployment. Kubernetes manifests provide namespace, configuration, secrets, persistent storage, workloads, services, and ingress resources for hosted deployment.

Documentation maintenance

Regenerate the images with docs/diagrams/regenerate_diagrams.py, then rebuild this PDF with docs/generate_system_documentation.py whenever roles, routes, data models, or deployment components change.